Comprehensive Study Guide

CME 295: Transformers & LLMs — Lecture 6

LLM Reasoning: Scaling with RL, GRPO, DeepSeek R1, Distillation

Stanford CME 295 by Afshine & Shervine Amidi · For: Applied AI/ML Engineers

Contents

Topic Map

1. **Vanilla LLM Limitations and Reasoning Overview**
   - 1.1 Four weaknesses: limited reasoning, static knowledge, no actions, hard to evaluate
   - 1.2 Definition of reasoning; reasoning vs. knowledge questions
   - 1.3 Core idea: Chain-of-Thought at scale
   - 1.4 Two intuitions for why more tokens = more reasoning
   - 1.5 Reasoning model timeline (o1, DeepSeek R1, Gemini Flash Thinking, etc.)
2. **Reasoning Benchmarks and Metrics**
   - 2.1 Coding benchmarks: HumanEval, CodeForces, SWE-bench; test-case verification
   - 2.2 Math benchmarks: AIME, GSM8K; answer parsing and ground-truth comparison
   - 2.3 Pass@k metric: definition, derivation, unbiased estimator
   - 2.4 Pass@1 as a special case
   - 2.5 Temperature and its role in pass@k
   - 2.6 Cons@k (Consensus@k) and its relation to self-consistency
3. **Scaling Reasoning with RL (Test-Time Scaling)**
   - 3.1 Why RL over SFT for reasoning
   - 3.2 Reward 1: formatting reward (think token delimiters)
   - 3.3 Reward 2: accuracy reward (verifiable, deterministic)
   - 3.4 Compute budget and test-time scaling
   - 3.5 Controlling thinking: dynamic budget, context awareness, budget forcing
   - 3.6 Continuous thoughts / latent reasoning
4. **GRPO: Group Relative Policy Optimization**
   - 4.1 GRPO vs. PPO: shared structure (ratio, clip), key differences
   - 4.2 Advantage estimation: group-relative vs. value-function-based
   - 4.3 Models required: GRPO (3: policy, reward, reference) vs. PPO (4: + value)
   - 4.4 GRPO loss formula; KL placement difference
   - 4.5 "Increasing output length" bias: the $1/|o_i|$ normalization problem
   - 4.6 Remedies: DAPO, Dr.GRPO
   - 4.7 Other adjustments: difficulty bias, diversity encouragement, asymmetric epsilon
5. **DeepSeek R1-Zero and R1 Training Pipelines**
   - 5.1 R1-Zero: base model → GRPO (verifiable rewards only)
   - 5.2 R1-Zero challenges: language mixing, formatting issues
   - 5.3 R1 full pipeline: 5-stage recipe
   - 5.4 Cold start data, rejection sampling, language consistency reward

Deep-Dive Notes

Vanilla LLM Limitations and the Reasoning Paradigm

Vanilla LLM Strengths and Weaknesses

Strengths:

- Excellent at imitation, idea generation, writing essays, poems.
- Remarkable at code generation and debugging.

Weaknesses (the four covered in this course):

1. **Limited reasoning:** Multi-step mathematical or logical problems rarely appear verbatim in training data; next-token prediction doesn't force systematic problem decomposition.
2. **Static knowledge:** Pre-training data has a cutoff date. Events after the cutoff are unknown from weights alone.
3. **Cannot perform actions:** Models only generate text; they cannot browse the web, execute code, or call APIs (covered in Lecture 7–8).
4. **Hard to evaluate:** Free-form text output doesn't fit classic NLP metrics like BLEU/ROUGE.

Definition: Reasoning

The ability to solve a problem through a **multi-step reasoning process**. Reasoning problems require decomposing a question into tractable sub-problems and chaining their solutions. Contrast with knowledge retrieval:

Knowledge question	Reasoning question
"What is CME 295?"	"A bear was born in 2020. How old is it now?"

Core Idea: Chain-of-Thought at Scale

Reasoning models extend Chain-of-Thought (introduced as a prompting technique in Lecture 3) from a prompt-level trick to a **training objective**. The model learns — through RL on verifiable rewards — to generate a reasoning chain before the final answer:

$$\text{Output} = \underbrace{\langle \texttt{think} \rangle \ldots \langle \texttt{/think} \rangle}_{\text{reasoning chain (hidden/summarized for users)}} + \underbrace{\text{answer}}_{\text{shown to user}}$$

Two Intuitions for Why More Tokens = Better Reasoning

1. **Decomposition:** Hard problems rarely appear in training data. By breaking a hard problem into tractable sub-problems, the model can apply patterns it *did* see during pre-training to each sub-problem.
2. **More compute:** Each generated token requires a full Transformer forward pass. Generating more reasoning tokens = more computation applied to the problem = effectively scaling inference-time compute.

This is called **test-time scaling**: improving performance by allocating more compute at inference time.

Reasoning Model Timeline (2024–2025)

- **Sep 2024:** OpenAI o1-preview — first public reasoning model.
- **Dec 2024:** Google Gemini 2.0 Flash Thinking.
- **Jan 2025:** DeepSeek R1 — open paper matching o1 performance; sparked massive community interest.
- **Feb 2025:** Claude 3.7 Sonnet (Anthropic); Grok 3 Beta (xAI).
- **Jun 2025:** Magistral (Mistral).

Key signal: when the UI says "thinking..." and shows a thought summary, you're using a reasoning model. You are charged for reasoning tokens (invisible to you but present in the generation).

Reasoning Benchmarks and Metrics

Benchmark Types

Coding Benchmarks

Given a problem statement, generate code that passes all t test cases. Correctness is fully automated and deterministic.

- **HumanEval:** 164 human-written Python problems.
- **CodeForces:** Competitive programming problems of varying difficulty.
- **SWE-bench:** Real GitHub issues requiring code fixes.

Math Benchmarks

Given a math problem, generate reasoning and a final answer. Correctness is verified by comparing the parsed answer to ground truth.

- **AIME:** American Invitational Mathematics Examination (olympiad-level).
- **GSM8K:** 8,500 grade-school math word problems (easier; used to track progress).

Answer parsing: prompt the model to wrap its final answer in a parseable format (e.g., `boxed{5}`).

Pass@k: Full Derivation

Pass@k: Definition

pass@k = the probability that at least one of k randomly sampled attempts solves the problem correctly.

Pass@k Derivation (Sampling Without Replacement)

Generate n total attempts, of which c succeed and $n-c$ fail. Select k of these n attempts uniformly at random. Estimate pass@k:

Step 1 — Complement:

$$\text{pass@}k = 1 - P(\text{all } k \text{ selected attempts fail})$$

Step 2 — Sampling without replacement probability:

$$P(\text{all } k \text{ fail}) = \frac{n-c}{n} \cdot \frac{n-c-1}{n-1} \cdots \frac{n-c-k+1}{n-k+1}$$

This is the product of probabilities that each of the k draws is a failure.

Step 3 — Express with combinatorics:

$$P(\text{all } k \text{ fail}) = \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

since $\prod_{i=0}^{k-1} \frac{n-c-i}{n-i} = \frac{(n-c)!/(n-c-k)!}{n!/(n-k)!} = \frac{\binom{n-c}{k}}{\binom{n}{k}}$.

Final formula:

$$\boxed{\widehat{\text{pass@}k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}}$$

Special case $k = 1$: pass@1 $= 1 - (n-c)/n = c/n$ (fraction of correct attempts). This is just accuracy.

Why Use This Formula Instead of Directly Measuring?

Directly counting pass rates from exactly k samples is high-variance (by chance, one run might get 3/5 correct, another 1/5). Generating $n > k$ samples and using this unbiased estimator reduces variance. The formula gives an unbiased estimate of the true probability by marginalizing over all possible k-subsets of the n samples.

Temperature and Pass@k

- $T = 0$ **(greedy):** All n attempts are identical. pass@k doesn't increase with k. No benefit from multiple samples.
- **Low** T **(e.g., 0.2–0.4):** High-quality but low-diversity samples. pass@k increases slowly with k.
- **Medium** T **(e.g., 0.8):** Good balance of quality and diversity. pass@k increases meaningfully with k.
- **High** T **(e.g., 1.2):** High diversity but lower individual quality. pass@k may plateau at a lower value.

Reported results in papers always specify the temperature used.

Cons@k (Consensus at k)

The majority-vote answer among k generations. Related to self-consistency (Lecture 3): generate k responses, parse the final answer from each, return the most common answer.

Scaling Reasoning with RL (Test-Time Scaling)

Why RL Over SFT for Reasoning?

Three Arguments for RL Over SFT in Reasoning

1. **No SFT data readily available:** Writing long, correct reasoning chains from scratch is extremely difficult and expensive. Human annotators make errors, especially in complex math/code.
2. **Human reasoning may not be optimal for models:** The way models reason internally may differ from how humans write explanations. Restricting to human-written chains could limit the model's discovered reasoning strategies.
3. **Verifiable rewards exist naturally:** For coding (test case pass/fail) and math (exact answer match), correctness is automatically and deterministically verifiable — no learned

reward model needed.

Two Reward Signals for Reasoning RL

Reward 1: Formatting Reward

Check that the model's output follows the required template — specifically, that reasoning is enclosed in `<think>...</think>` tags and the answer follows in `<answer>...</answer>` tags. Binary: present or not.

Reward 2: Accuracy Reward (Verifiable)

For **coding:** does the generated code pass all test cases?
For **math:** does the parsed answer match the ground truth?
These rewards are **deterministic and automatic** — no reward model required. This is a major practical advantage over preference-tuning RLHF where the reward model must itself be trained.
Total reward = formatting reward + accuracy reward.

Controlling Thinking at Inference Time

Test-Time Scaling: Controlling Compute Budget

Not all prompts require the same depth of thinking. Methods to control thinking duration:

- **Dynamic budget:** Use a lightweight classifier to estimate problem difficulty and allocate different token budgets accordingly.
- **Context awareness:** Force the model to be aware of its remaining context window; stop thinking when approaching the limit.
- **Budget forcing (s1 paper):** Insert special tokens mid-reasoning to force the model to continue (e.g., "wait, let me reconsider...") or stop (e.g., "my final answer is...").
- **Continuous thoughts:** Instead of reasoning in text tokens, have the model reason in continuous hidden-state representations (latent CoT). More compact and possibly richer than text.

GRPO: Group Relative Policy Optimization

GRPO (Shao et al., 2024 — DeepSeekMath)

An RL algorithm for LLMs that replaces PPO's value-function-based advantage estimation with a group-relative advantage:

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

where $r_1, \ldots, r_G$ are rewards for G completions sampled from the *same prompt*. The advantage of completion i is how much better it is than the group average, in units of standard deviation.

Table 1: GRPO vs. PPO Comparison

Dimension	GRPO	PPO
Advantage estimation	Group-relative: $\frac{r_i - \mu_G}{\sigma_G}$	GAE: reward + value function
Value function	Not needed	Trained jointly with policy
Models needed	3 (policy, reward, reference)	4 (+ value model)
KL divergence	Explicit in loss formula	Inside advantage (per-token reward)
Policy ratio	$\pi_\theta / \pi_{\theta_{\text{old}}}$	$\pi_\theta / \pi_{\theta_{\text{old}}}$
Clipping	Yes (ε)	Yes (ε)

GRPO vs. PPO: Side-by-Side

GRPO Loss Formula

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q,\{o_i\}}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\left(r_t^{(i)}(\theta)\hat{A}_i,\ \text{clip}(r_t^{(i)}(\theta), 1\pm\varepsilon)\hat{A}_i\right) - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})\right]$$

where $r_t^{(i)}(\theta) = \pi_\theta(o_t^{(i)}|q, o_{<t}^{(i)}) / \pi_{\theta_{\text{old}}}(o_t^{(i)}|q, o_{<t}^{(i)})$ is the token-level policy ratio, $|o_i|$ is the length of completion i, and $\hat{A}_i$ is the group-relative advantage.

Key structural note: The outer sum is over completions $i \in \{1, \ldots, G\}$; the inner sum is over token positions $t \in \{1, \ldots, |o_i|\}$. The $1/|o_i|$ normalizes by completion length. The KL term is explicit in the GRPO loss (outside the summations), whereas in PPO it is embedded within the per-token rewards.

The Increasing Output Length Problem

The $1/|o_i|$ Bias: A Bad Incentive

The normalization by length $1/|o_i|$ in the GRPO loss creates an unintended bias. Consider token t in completion i:

Weight of token t's contribution to the objective:

$$w_t = \frac{1}{|o_i|} \times (\text{token-specific objective term})$$

- For a **short** completion: $1/|o_i|$ is *large* $\Rightarrow$ each token has higher weight.
- For a **long** completion: $1/|o_i|$ is *small* $\Rightarrow$ each token has lower weight.

The problem: When advantage $\hat{A}_i < 0$ (bad completion), the optimizer wants to *decrease* the probability of the tokens in that completion. The gradient to do so is:

- Stronger for tokens in **short bad** completions (larger weight).
- Weaker for tokens in **long bad** completions (smaller weight).

This means: **a short bad completion is penalized more than a long bad completion**. The policy learns to prefer longer outputs even when wrong, because longer bad completions receive weaker penalty signals. This causes the empirically observed "output length keeps growing" phenomenon during RL training.

Remedies: DAPO and Dr.GRPO (2025)

- **DAPO (Yu et al., 2025):** Replace $1/|o_i|$ with a common normalization factor across all tokens in all completions: $\sum_{i,t} 1$ (total token count). This equalizes the per-token contribution regardless of completion length.
- **Dr.GRPO (Liu et al., 2025):** Remove the $1/|o_i|$ normalization factor entirely (replace with 1). Also: filter out examples where all completions in the group are correct or all are wrong (zero-variance groups).

Empirical result: With these fixes, incorrect completions have shorter average length (no more length inflation), while correct completions maintain similar length. Performance improves.

Other GRPO Adjustments

- **Difficulty bias from std:** If a problem is very hard (most completions fail), the standard deviation of rewards is small (near-zero). This makes the normalized advantages very large, producing unstable gradients. Solution: add a small constant to the denominator or filter out degenerate groups.
- **Diversity encouragement (DAPO):** Use asymmetric clipping: a larger upper ε for increasing token probabilities and a smaller lower ε for decreasing them. Encourages the policy to explore new tokens without catastrophically forgetting old ones.

DeepSeek R1-Zero and R1 Training Pipelines

DeepSeek R1-Zero (Proof of Concept)

R1-Zero Recipe (2 stages)

Stage 1 — Pre-training: V3-Base. Standard next-token prediction pre-training. Architecture: MoE with ∼671B total parameters, ∼37B active per token (using MLA from DeepSeek-V2).

Stage 2 — GRPO with verifiable rewards only (no SFT):

- Reward = formatting reward + accuracy reward.
- Template: instructs model to wrap reasoning in `<think>...</think>` and answer in `<answer>...</answer>`.
- Result: reasoning abilities emerge purely from RL, without any supervised reasoning demonstrations.

Outcome: AIME performance improves substantially over training steps.

Challenges: Reasoning chains sometimes mix languages (e.g., Chinese and English in the same chain); formatting and readability issues.

DeepSeek R1 (Full Production Pipeline)

R1 Full Recipe (5 stages)

Starting from V3-Base:

Stage 1 — Pre-training: V3-Base (same as R1-Zero).

Stage 2 — Small-scale cold start SFT:

- Data: long CoTs generated by R1-Zero, then rewritten by humans to fix language mixing, formatting, and readability issues.
- Scale: small (∼ thousands of examples). Provides a clean starting point before the full

RL stage.

Stage 3 — GRPO with reasoning data:

- Same RL process as R1-Zero but starting from the cold-start SFT checkpoint.
- Reward = formatting + accuracy + **language consistency** (ratio of target-language tokens in the reasoning chain — penalizes language mixing).

Stage 4 — Large-scale SFT with mixed data:

- ∼600K reasoning pairs (math, coding, logic): generated by the model-so-far via **rejection sampling** (generate many responses, keep only those verified as correct by rules + V3-judge).
- ∼200K non-reasoning pairs: reused from V3's SFT data (writing, Q&A, general assistant tasks).
- Ratio approximately 3:1 (reasoning:non-reasoning).

Stage 5 — GRPO with mixed reasoning and non-reasoning data: R1

- Reasoning data: reward = formatting + accuracy.
- Non-reasoning data: reward = helpfulness + harmlessness (similar to standard RLHF).
- Harmlessness reward applied to the full output including the thinking chain.

Rejection Sampling (for SFT data generation)

A technique for generating high-quality training data: have the model (or a strong LLM) produce many candidate responses, then use automated verification (test cases, answer comparison) and/or an LLM judge to keep only the correct/high-quality ones. The resulting dataset is used for SFT training. More efficient than human annotation at scale.

Distillation of Reasoning Capabilities

Two Flavors of Distillation

Lecture 2 (KD-style) Distillation	Reasoning Distillation (R1-Distill)
Goal: match the full token probability distribution of the teacher at each step	Goal: predict the full token *sequence* generated by the teacher
Online: teacher scores each student prediction	Offline: teacher generates responses first, student SFT-learns them
Loss: KL divergence of teacher vs. student distributions	Loss: cross-entropy (next-token prediction on teacher's sequences)

R1-Distill: Offline Reasoning Distillation

Step 1 (offline): Use R1 (teacher, 671B) to generate complete responses including `<think>...</think>` reasoning traces for a set of prompts.

Step 2 (training): Fine-tune a smaller student model (e.g., LLaMA 3-8B, Qwen-7B) via standard SFT on these (prompt, full-response) pairs. The student learns to mimic the entire reasoning sequence.

This is "distillation" in the broader sense: transferring reasoning ability from a large model to a small one via its outputs.

Why Distillation > RL from Scratch at Small Scale

At small model sizes (7B–14B parameters), directly applying GRPO with verifiable rewards produces much weaker reasoning than distilling from R1. Why:

- Small models have limited capacity to discover effective reasoning strategies from scratch through RL exploration.
- The teacher's reasoning traces provide rich, complete demonstrations that SFT can learn efficiently.
- RL from scratch requires many rollouts to discover good solutions; small models waste much of this compute on unhelpful exploration.

Empirical result: R1-Distill-7B is competitive with o1-mini on several benchmarks; much better than training a 7B model with RL from scratch.

Related Concepts You Should Also Know

1. **Test-Time Compute Scaling:** The phenomenon that allocating more inference-time compute (more reasoning tokens) improves performance. Analogous to training-time scaling laws. Sometimes called "inference scaling." Key result: reasoning models with 8× more compute can match models with 10× more parameters on some benchmarks.
2. **REINFORCE vs. GRPO vs. PPO:** REINFORCE uses the raw return (reward) as the advantage, with no baseline. High variance. GRPO uses the group-mean-subtracted reward as the advantage — a natural baseline that reduces variance without needing a value function. PPO uses the GAE baseline from a jointly trained value function — lower variance but requires the extra model.
3. **Process Reward Models (PRMs):** Instead of rewarding only the final answer (Outcome Reward Model, ORM), PRMs reward correctness at each reasoning step. PRMs can provide denser training signals but are more expensive to collect labels for (each reasoning step must be labeled, not just the final answer).
4. **Self-Play and Outcome-Based RL:** GRPO with verifiable rewards is a form of self-play where the model generates its own training data (completions) and learns from their correctness. This is how AlphaGo-style systems work, adapted to language.
5. **Search-Based Inference (MCTS for LLMs):** Some approaches use Monte Carlo Tree Search at inference time to explore the reasoning tree and select high-reward paths. More expensive than sampling but can find better solutions.
6. **"Aha Moment" in R1-Zero:** During R1-Zero's RL training, the researchers observed the model spontaneously develop self-checking behavior ("wait, let me reconsider"), even without being taught to do so. This is interpreted as an emergent behavior arising from the RL optimization incentive.
7. **Budget Forcing (s1 paper, Muennighoff et al., 2025):** A simple approach to test-time compute scaling: when the model generates an end-of-thinking token, inject "Wait" to force it to continue reasoning. This can improve pass@1 with minimal engineering overhead.
8. **Latent (Continuous) Chain-of-Thought:** Instead of reasoning in discrete text tokens, have the model reason in its hidden-state space. The reasoning "tokens" are continuous vectors rather than words. More compact and potentially more expressive. Active area of research (Hao et al., 2024; ongoing).
9. **SWE-bench Verified:** A cleaner version of SWE-bench where problem statements have been human-validated for unambiguity. Important for reliable benchmarking of code reasoning.
10. **Overfitting to Reasoning Benchmarks:** As with general benchmarks (Lecture 4), models can overfit to reasoning benchmarks if trained on similar problems. Valid reasoning evaluation requires held-out problem distributions not seen during RL training.

Build Projects

1. **Pass@k Estimator and Temperature Analysis** *(2–3 hours, good signal)*
   Using a small open-source model (e.g., CodeLlama-7B or Qwen-Coder-7B) and a subset of HumanEval (20 problems), implement the pass@k estimator from scratch using the exact unbiased formula derived in class. Generate $n = 50$ solutions per problem at temperatures $T \in \{0, 0.2, 0.4, 0.8, 1.2\}$. Plot pass@k curves ($k \in \{1, 5, 10, 20\}$) for each temperature. Verify empirically that moderate temperature maximizes pass@k for large k. This is a complete, self-contained experiment that tests your understanding of both the metric and temperature's role.

2. **GRPO from Scratch on a Math Task** *(6–10 hours, very high signal)*
   Implement GRPO on a small (1B–3B) language model for a simple math task (e.g., single-digit arithmetic, modular arithmetic). Key components: (a) generate $G = 4$ completions per prompt, (b) compute verifiable reward (exact answer match), (c) compute group-relative advantages, (d) apply the PPO-Clip objective with GRPO's group structure, (e) add KL penalty against the frozen reference model. Track: average reward, response length, pass@1. Then apply the Dr.GRPO fix (remove $1/|o_i|$) and compare length evolution. This is a direct implementation of the core algorithm from this lecture.

3. **Reasoning Distillation Pipeline** *(4–6 hours, high signal)*
   Using the HuggingFace TRL library and an open-source reasoning model (e.g., DeepSeek-R1-Distill-7B or Qwen2.5-Math-7B), generate reasoning traces for 500 GSM8K problems. Use the traces that produce correct answers (rejection sampling). Fine-tune a smaller base model (e.g., LLaMA 3.2-3B) on these (problem, full-trace) pairs via SFT. Compare pass@1 on a held-out GSM8K split before and after distillation fine-tuning. Compare with the 3B model trained with RL from scratch (GRPO) on the same problems.

Explain It Back

1. Explain to a junior engineer why reasoning models need RL rather than SFT. Cover all three arguments (data availability, human vs. model reasoning style, verifiable rewards). Then explain why verifiable rewards are a practically important advantage — what would you need instead if you didn't have them?
2. Derive the pass@k formula from first principles. Start by explaining why you generate $n > k$ samples, then apply the complement trick, then the sampling-without-replacement argument. Verify that when $k = 1$ the formula simplifies to c/n.
3. Explain the output-length bias in GRPO's $1/|o_i|$ normalization. Walk through a concrete example: suppose you have two incorrect completions, one 10 tokens long and one 100 tokens long. How does the gradient magnitude for each token differ between the two, and why does this incentivize the model to generate longer outputs?
4. Describe the DeepSeek R1 training pipeline in 5 sentences or fewer (one per stage). For each stage, state what data is used, what the training objective is, and what problem it solves.

Interview Practice Questions & Answers

L1 — What is GRPO and how does its advantage estimation differ from PPO?

Answer: GRPO (Group Relative Policy Optimization) is an RL algorithm for LLM training that avoids the value function required by PPO. For a given prompt, GRPO samples G completions, computes a reward for each, and defines the advantage of completion i as $(r_i - \mu_G)/\sigma_G$ — how many standard deviations above or below the group mean its reward is. This group-relative advantage replaces the value-function-based GAE in PPO. The benefit: GRPO needs only 3 models (policy, reward, reference) vs. PPO's 4 (adds a value model trained jointly with the policy). The tradeoff: GRPO requires more completions per prompt (sampling group-level variance as the baseline), while PPO amortizes the baseline across the entire training distribution via the value function.

L1 — Why are verifiable rewards important for reasoning RL?

Answer: Standard preference-tuning RLHF requires training a reward model (RM) from human preference labels. This involves two stages (RM training, then RL), requires tens of thousands of human labels, and the RM itself is imperfect (reward hacking). For coding and math reasoning, correctness is verifiable automatically: code either passes test cases or it doesn't; a math answer either matches the ground truth or it doesn't. This eliminates the need for a reward model entirely, making the reward signal (a) free to compute, (b) perfectly accurate (no reward hacking from RM imperfections), and (c) available at unlimited scale (generate as many problems as needed).

L2 — Explain the output-length inflation problem in GRPO and how DAPO addresses it.

Answer: The GRPO loss sums per-token objectives weighted by $1/|o_i|$, where $|o_i|$ is the length of completion i. A token in a short bad completion receives weight 1/10 (for a 10-token output) vs. 1/100 for the same quality token in a 100-token output. Since the optimizer tries to reduce the probability of tokens with negative advantage, it applies 10× stronger downward pressure on the short bad completion than the long one. This makes long incorrect completions "cheaper" to produce than short ones, biasing the policy toward longer outputs even when they're wrong. Empirically, this causes response length to keep growing throughout RL training even after performance plateaus. DAPO addresses this by replacing $1/|o_i|$ with a global normalization constant (total tokens in the batch), so every token regardless of completion length gets equal weight.

L2 — Describe the DeepSeek R1 full training pipeline and the purpose of each stage.

Answer: R1 uses a 5-stage pipeline starting from V3-Base (pre-trained MoE). Stage 2 is a small-scale cold-start SFT on human-corrected reasoning chains from R1-Zero, fixing language mixing and formatting issues. Stage 3 is GRPO with formatting + accuracy + language consistency rewards, building reasoning capability from the cleaner starting point. Stage 4 is a large-scale SFT on 600K rejection-sampled reasoning pairs (verified by rules and V3-judge) plus 200K non-reasoning pairs, making the model both highly capable at reasoning and useful as a general assistant. Stage 5 is a final GRPO pass on mixed reasoning and non-reasoning data, with helpfulness and harmlessness rewards for the non-reasoning component, ensuring the final model is safe and helpful in addition to being a strong reasoner.

L3 — Derive the unbiased pass@k estimator from sampling without replacement.

Answer: Generate n solutions, c correct and $n - c$ incorrect. Want $P(\text{at least 1 of } k \text{ randomly selected solutions is correct})$. By complementarity: $\text{pass@}k = 1 - P(\text{all } k \text{ selected are incorrect})$. The probability that the first selected is incorrect is $(n-c)/n$, the second $(n-c-1)/(n-1)$, etc., giving $\prod_{j=0}^{k-1}(n-c-j)/(n-j)$. This product equals $\binom{n-c}{k}/\binom{n}{k}$ (the numerator is the number of ways to choose k failures from $n-c$ failures; the denominator is all ways to choose k from n). So $\widehat{\text{pass@}k} = 1 - \binom{n-c}{k}/\binom{n}{k}$. When $k = 1$: $1 - (n-c)/n = c/n$, which is just the fraction of correct solutions.

L2 — Why is distillation more effective than RL from scratch for small (7B) reasoning models?

Answer: RL from scratch on small models faces two problems: (1) the model has limited capacity to discover effective long-form reasoning strategies through random exploration — most GRPO rollouts produce incorrect solutions, providing weak learning signal; (2) the policy must simultaneously learn *how* to reason (chain structure, reflection, backtracking) and *what* the correct reasoning looks like for each problem, making the RL optimization landscape difficult to navigate. Distillation bypasses both issues: the teacher (R1) provides complete, correct reasoning traces that the student SFT-learns directly. This is equivalent to giving the small model a rich curriculum of solved examples. Empirically, DeepSeek's R1-Distill-7B outperforms a 7B model trained with GRPO from scratch on most benchmarks.

L1 — What is test-time scaling and why does it work?

Answer: Test-time scaling is the strategy of improving model performance at inference time by generating more reasoning tokens before producing the final answer. Each reasoning token requires a full Transformer forward pass, so more reasoning tokens = more computation applied to the problem. This works because: (1) hard problems can be decomposed into sub-problems each of which may match training examples, (2) more compute allows the model to detect and correct intermediate errors (self-reflection), and (3) the model can explore multiple reasoning paths and select the most consistent one. In practice, reasoning models can match or exceed larger dense models on reasoning tasks by spending more inference-time compute, making it possible to improve performance without increasing model parameters.

Self-Assessment Test

Scoring: 16–20 = solid mastery; 12–15 = revisit weak spots; <12 = re-study.

Multiple Choice (1 point each)

Q1. What is the key difference between GRPO and PPO in terms of models needed?
(a) GRPO needs 5 models; PPO needs 3 (b) GRPO eliminates the value model; PPO requires it (c) GRPO requires a reward model; PPO does not (d) There is no difference

Q2. Pass@k with $k = 1$ and $n = 20$ samples, 4 of which are correct, equals:
(a) $4/20 = 0.2$ (b) 1 - $16/20 = 0.2$ (c) $1 - \binom{16}{1}/\binom{20}{1} = 0.2$ (d) All of the above

Q3. In R1-Zero, what was the key challenge observed after GRPO training?
(a) The model refused to produce reasoning chains (b) Reasoning chains mixed languages and had formatting issues (c) The accuracy reward was too sparse to learn from (d) The model became shorter in its responses

Q4. The $1/|o_i|$ normalization in GRPO causes what problem?
(a) Short correct completions are not rewarded enough (b) Long incorrect completions are penalized less than short incorrect ones, incentivizing longer outputs (c) The KL divergence grows unbounded (d) The group advantage cannot be computed

Q5. In the DeepSeek R1 pipeline, what is the purpose of Stage 2 (cold start SFT)?
(a) Teach the model arithmetic from scratch (b) Fix language mixing and formatting issues observed in R1-Zero before the main RL stage (c) Replace the GRPO stage entirely (d) Generate the non-reasoning SFT data

Q6. Why is temperature $T = 0$ problematic for pass@k evaluation with $k > 1$?
(a) It produces very long outputs (b) All n samples are identical, providing no diversity; pass@k doesn't improve with more samples (c) It crashes the model (d) It underestimates pass@1

Q7. What is Cons@k (Consensus@k)?
(a) The average reward across k completions (b) The majority-vote answer among k generations (c) The pass@k estimator with $n = k$ (d) The number of successful completions in k tries

Q8. R1-Distill outperforms RL-from-scratch at small model sizes because:
(a) It uses more compute (b) Small models can more efficiently SFT-learn rich reasoning traces from R1 than discover them through RL exploration (c) The distillation loss is easier to optimize than GRPO (d) It doesn't require any data

Short Answer (1.5 points each)

Q9. State the unbiased pass@k formula and explain in one sentence what "sampling without replacement" has to do with it.

Q10. What are the two reward signals used in GRPO-based reasoning RL training? Why is no separate reward model needed?

Q11. Explain the group-relative advantage in GRPO. How many completions are needed per prompt, and what is the advantage formula?

Q12. What is rejection sampling in the context of the R1 training pipeline, and what is it used for?

Q13. How does reasoning distillation (R1-Distill) differ from the KD-style distillation covered in Lecture 2?

Q14. Explain budget forcing. Under what circumstances would you use it?

Scenario (2 points)

Q15. Your team wants to build a coding assistant that solves competitive programming problems (CodeForces difficulty). You have a pre-trained 70B LLM (instruction-tuned, but not reasoning-tuned), access to CodeForces problems with test cases, and 8 A100-80GB GPUs. Describe a complete training approach to make this model competitive on CodeForces using RL. Cover: (a) choice of RL algorithm (GRPO or PPO) and why, (b) reward design, (c) training data sourcing, (d) how you would evaluate progress (metrics and methodology), (e) one practical concern about the training process and how you'd mitigate it.

Answer Key

Q1. (b) GRPO eliminates the value function model, requiring 3 models; PPO requires 4.

Q2. (d) All of the above are equivalent expressions of $c/n = 4/20 = 0.2$.

Q3. (b) R1-Zero's reasoning chains sometimes mixed languages (e.g., Chinese and English) and had formatting/readability issues.

Q4. (b) Tokens in long incorrect completions get smaller per-token gradient magnitude (weight $= 1/|o_i|$) than tokens in short incorrect completions, so the model learns that longer bad outputs are less "bad."

Q5. (b) Cold start SFT on human-corrected reasoning chains fixes the language mixing and formatting problems from R1-Zero before the main RL training stage.

Q6. (b) With $T = 0$, all n generated solutions are identical (deterministic), so c is either n (if the greedy answer is correct) or 0. There is no diversity, and pass@k equals pass@1 for all k.

Q7. (b) Cons@k returns the answer that appears most frequently (majority vote) across k generated solutions.

Q8. (b) At small model sizes, RL exploration is inefficient — the model rarely finds correct solutions on its own. Distillation from R1's rich correct traces provides dense supervision.

Q9. $\widehat{\text{pass@}k} = 1 - \binom{n-c}{k}/\binom{n}{k}$, where n = total samples, c = correct samples. The sampling-without-replacement connection: the denominator $\binom{n}{k}$ counts all ways to select k from n samples; the numerator $\binom{n-c}{k}$ counts ways to select k from only the $n - c$ incorrect samples; their ratio is the probability that a random k-subset contains no correct solutions.

Q10. Reward 1: formatting reward (checks for `<think>...</think>` and `<answer>...</answer>` delimiters). Reward 2: accuracy reward (code passes all test cases; math answer matches ground truth). No separate reward model is needed because correctness is deterministically and automatically verifiable from the problem specification.

Q11. For each prompt, sample G completions (typically $G \in \{4, 8, 16\}$). Compute reward r_i for each. The group-relative advantage for completion i is: $\hat{A}_i = (r_i - \mu_G)/\sigma_G$ where $\mu_G = \frac{1}{G}\sum r_j$ and σ_G is the group standard deviation.

Q12. Rejection sampling: generate many candidate responses using the current model checkpoint, verify them automatically (test cases for code, answer comparison for math, or LLM judge for general quality), and keep only the verified-correct ones for the SFT dataset. Used in Stage 4 of R1's pipeline to create 600K high-quality reasoning SFT pairs.

Q13. Lecture 2 KD: teacher and student run simultaneously; student minimizes KL between its token distribution and the teacher's at each position; online process. R1-Distill: teacher generates complete responses offline first; student then SFT-learns to predict those complete sequences via standard next-token prediction; offline process. R1-Distill transfers complete reasoning strategy (including think tokens); KD transfers the probability distribution at each token position.

Q14. Budget forcing: inject "Wait" (or similar) tokens when the model tries to end its reasoning chain, forcing it to continue. Used when: (a) the default reasoning budget is too short for a hard problem, (b) you want to allocate more inference compute to improve accuracy at the cost of latency, (c) evaluating how much performance can be extracted with extended reasoning.

Q15. (a) **GRPO.** Reasons: no value function needed (3 models vs. 4 — critical with only 8 GPUs), verifiable rewards (test cases) eliminate need for learned RM, and GRPO is the standard for reasoning RL. (b) **Reward:** binary accuracy reward (all test cases pass = 1,

else 0) + formatting reward (if using think tokens). No RM needed. Apply DAPO fix (remove $1/|o_i|$) to prevent output length inflation. (c) **Training data:** CodeForces problems with public test cases + hidden test cases from a local judge. Source: CodeForces public API, filtered by difficulty rating. (d) **Evaluation:** pass@1 on a held-out set of CodeForces problems not used in training. Use $T = 0.8$ for pass@k with $k > 1$. Track: pass@1, pass@5, average response length, average reward per iteration. (e) **Practical concern:** Output length inflation — the model generates progressively longer (but not more correct) solutions. Mitigation: apply DAPO's global normalization (remove $1/|o_i|$); monitor average length of incorrect vs. correct solutions separately; if incorrect solutions remain long, add a length penalty term to the reward.

End of Study Guide — Lecture 6

Source: Stanford CME 295, Fall 2025. Afshine & Shervine Amidi.